JUnit5 Assertions

- Assertions are used to verify expected outcomes in tests.
- If an assertion fails, the test fails.

Let's go through each of these assertions one by one:

1. **assertEquals():**

- Checks if two values are equal or not.
- If expected and actual value matches, test case is considered PASS.

  assertEquals(expected, actual);

  assertEquals(expected, actual, failureMessage);

```java
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class MyServiceTest {

    @Test
    void testAddition() {

        // arrange
        MyService obj = new MyService();

        // act
        int output = obj.multiply(5, 6);

        // assert
        assertEquals(30, output, "both are not equal");
    }
}
```

For primitive values it uses "==" and for objects it uses ".equals()"

So, if you are comparing custom objects, then do override "equals" and "hashCode" method in your class.

else

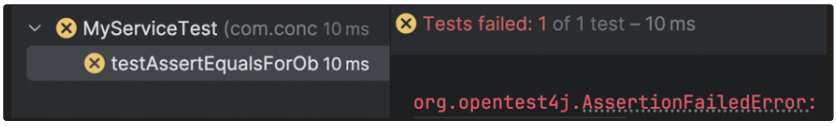
Concept && Coding

Default "equals" method from Object class will be used. Which checks for reference equality means, 2 different object with same value won't be considered same.

Custom Object without overriding "equals" method:

```java
public class MyService {

    String name;

    public MyService(String name) {
        this.name = name;
    }
}
```

```java
class MyServiceTest {

    @Test
    void testAssertEqualsForObject() {

        MyService object1 = new MyService("XYZ");
        MyService object2 = new MyService("XYZ");

        assertEquals(object1, object2);
    }
}
```



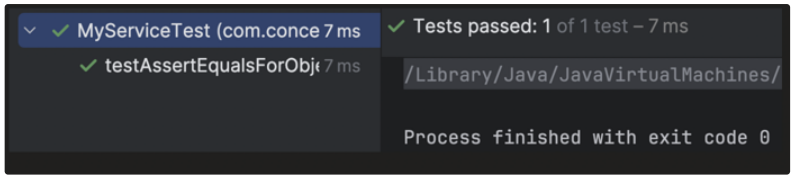
Custom Object with overriding "equals" method:

```java
import java.util.Objects;

public class MyService {

    String name;

    public MyService(String name) {
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof MyService)) return false;

        MyService servObject = (MyService) o;
        return Objects.equals(this.name, servObject.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }
}

```

```java
import static org.junit.jupiter.api.Assertions.assertEquals;
```

Concept && Coding

```java
import org.junit.jupiter.api.Test;

class MyServiceTest {

    @Test
    void testAssertEqualsForObject() {

        MyService object1 = new MyService("XYZ");
        MyService object2 = new MyService("XYZ");

        assertEquals(object1, object2);
    }
}

```



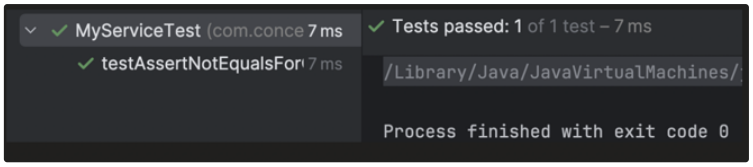
2. **assertNotEquals():**

- Checks if two values are not equal.
- If unexpected and actual value do not matches, test case is considered PASS.

assertNotEquals(unexpected, actual);

```java
@Test
void testAssertNotEqualsForObject() {

    MyService object = new MyService();

    int actualResult = object.multiply( a: 5, b: 6); //30
    assertNotEquals( unexpected: 11, actualResult);
}
```

Output:



3. **assertArrayEquals()**

Verifies 2 arrays contain the same element in same order.

Concept && Coding

```java
@Test
void testAssertArrayEquals() {

    int[] expected = {1, 2, 3};
    int[] actual = MyService.sortArray(new int[]{3, 1, 2});
    assertArrayEquals(expected, actual);  //for primitive: 1==1, 2==2, 3==3
}
```

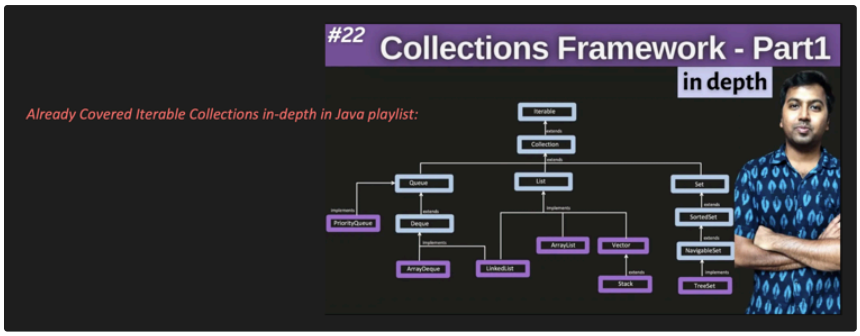
```
✓ MyServiceTest (com.conce 7 ms        ✓ Tests passed: 1 of 1 test – 7 ms
    ✓ testAssertArrayEquals() 7 ms     /Library/Java/JavaVirtualMachines/

                                       Process finished with exit code 0
```

```java
@Test
void testAssertArrayEquals() {

    String[] arr1 = {"Hello", "World"};
    String[] arr2 = {new String( original: "Hello"), new String( original: "World")};

    assertArrayEquals(arr1, arr2); //for object: "hi".equals("hi"), "bye".equals("bye")
}
```

```
✓ MyServiceTest (com.conce 7 ms        ✓ Tests passed: 1 of 1 test – 7 ms
    ✓ testAssertArrayEquals() 7 ms     /Library/Java/JavaVirtualMachines/

                                       Process finished with exit code 0
```

4. **assertIterableEquals()**

- Similar to "assertArrayEquals", but its used to compare two Iterable collections like List, Queue etc.
- Checks element by element in order using ".equals()"
- Iteration sequence should match.
- "Set" do not guarantee insertion order, so its not recommended to use this assertion on such unordered collections.



Concept && Coding

```java
@Test
void testAssertIterableEquals() {

    List<String> expected = List.of("A", "B", "C");
    List<String> actual   = List.of("A", "B", "C");

    assertIterableEquals(expected, actual);
}
```

```
MyServiceTest (com.conce 7 ms
    testAssertIterableEquals 7 ms
Tests passed: 1 of 1 test – 7 ms
/Library/Java/JavaVirtualMachines/jdk-17

Process finished with exit code 0
```

5. **assertLinesMatch()**

- Both expected and actual must be List<String>
- Each element of List<String> is treated as one line.
- Supports regex match too.

assertLinesMatch(List<String> expectedLines, List<String> actualLines)

```java
@Test
void testAssertLinesMatch() {

    List<String> expected = List.of("Hello", "World");
    List<String> actual   = List.of("Hello", "World");

    assertLinesMatch(expected, actual);
}
```

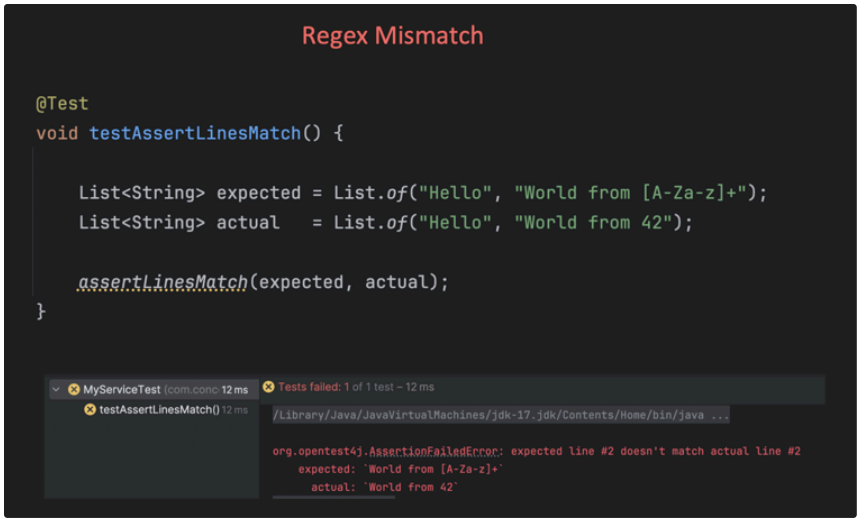
```
MyServiceTest (com.conce 7 ms
    testAssertLinesMatch() 7 ms
Tests passed: 1 of 1 test – 7 ms
/Library/Java/JavaVirtualMachines/jdk-17.jdk/

Process finished with exit code 0
```

Regex matches

```java
@Test
void testAssertLinesMatch() {

    List<String> expected = List.of("Hello", "World from [A-Za-z]+");
    List<String> actual   = List.of("Hello", "World from XYZ");

    assertLinesMatch(expected, actual);
}
```

```
MyServiceTest (com.conce 7 ms
    testAssertLinesMatch() 7 ms
Tests passed: 1 of 1 test – 7 ms
/Library/Java/JavaVirtualMachines/jdk-17.jdk/

Process finished with exit code 0
```

Concept && Coding



Regex Patterns:

- **Digits** → \\d+
  (e.g., "123", "42")

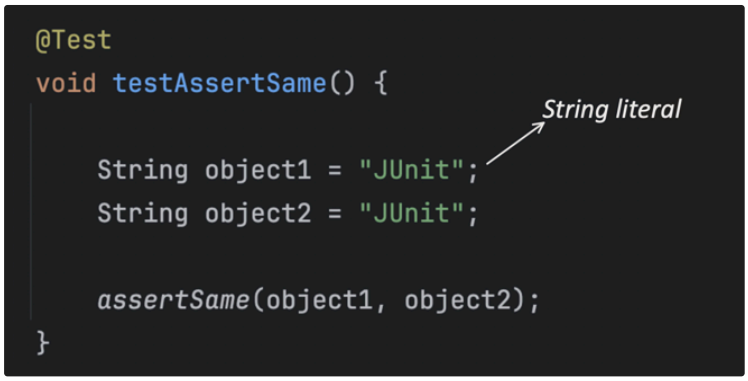
- **Letters/words** → [A-Za-z]+
  (e.g., "Hello", "World")

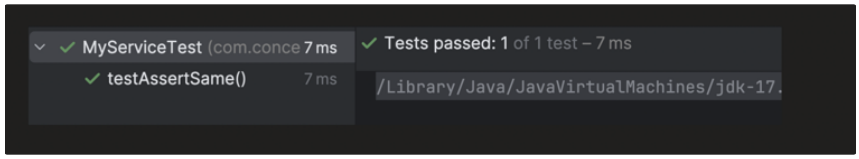
- **Alphanumeric** → \\w+
  (same as [A-Za-z0-9_], includes underscore)

- **Anything (any characters)** → .*
  (matches whole line, useful if only part matters)

6. **assertSame():**

Just Checks does both objects point to same reference or not.





Concept && Coding

in below example, both object1 and object2 points to different object:

```
@Test
void testAssertSame() {

    String object1 = "JUnit";
    String object2 = new String( original: "JUnit");

    assertSame(object1, object2);
}
```

```
MyServiceTest (com.conce 9 ms        Tests failed: 1 of 1 test – 9 ms
    testAssertSame()     9 ms        /Library/Java/JavaVirtualMachines/jd

                                     org.opentest4j.AssertionFailedError:
                                     <6 internal lines>
```

7. **assertNotSame():**

Checks if both the objects has difference reference.

```
@Test
void testAssertNotSame() {

    String object1 = "JUnit";
    String object2 = new String( original: "JUnit");

    assertNotSame(object1, object2);
}
```

```
MyServiceTest (com.conce 7 ms        Tests passed: 1 of 1 test – 7 ms
    testAssertNotSame()  7 ms        /Library/Java/JavaVirtualMachines/
```

8. **assertNull():**

Checks if object value is Null.

Concept && Coding

```
@Test
void testAssertNull() {

    MyService object= null;
    assertNull(object);
}
```

MyServiceTest (com.conce 6 ms
testAssertNull() 6 ms
Tests passed: 1 of 1 test – 6 ms
/Library/Java/JavaVirtualMachines/

9. **assertNotNull()**

Checks if object value is not Null.

```
@Test
void testAssertNotNull() {

    MyService object= new MyService();
    assertNotNull(object);
}
```

MyServiceTest (com.conce 6 ms
testAssertNotNull() 6 ms
Tests passed: 1 of 1 test – 6 ms
/Library/Java/JavaVirtualMachines/

10. **assertTrue()**

Check if condition is True.

```
@Test
void testAssertTrue() {

    assertTrue( condition: 5>2);
}
```

MyServiceTest (com.conce 6 ms
testAssertTrue() 6 ms
Tests passed: 1 of 1 test – 6 ms
/Library/Java/JavaVirtualMachines/jdk

11. **assertFalse()**

Check if condition is False.

Concept && Coding

```
@Test
void testAssertFalse() {

    assertFalse( condition: 5<2);
}
```

```
MyServiceTest (com.conce 6 ms        Tests passed: 1 of 1 test – 6 ms
  testAssertFalse()      6 ms        /Library/Java/JavaVirtualMachines/jdk-17.
```

12. **fail()**

- It fails the test deliberately.
- Generally used when flow reach to the point where it should not.

```
@Test
void testAssertFail() {

    MyService object = new MyService();
    try {
        object.methodThrowsException();
        fail("above method should trow exception");
    }
    catch (Exception e) {
        //expecting the control should reach here.
    }
}
```

```
MyServiceTest (com.conc 10 ms        Tests failed: 1 of 1 test – 10 ms
  testAssertFail()      10 ms        /Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home/bin/java ...

                                     org.opentest4j.AssertionFailedError: above method should trow exception
                                     <2 internal lines>
```

13. **assertInstanceOf()**

Check if object is an instance of specified class (or subclass)

```
@Test
void testAssertInstanceOf() {

    Number n = Integer.valueOf( i: 10);

    // passes, because Integer is a subclass of Number
    assertInstanceOf(Number.class, n);

    // fails, because n is not a Double
    assertInstanceOf(Double.class, n);
}
```

Concept && Coding

14. **assertThrows()**

- If the method throws the expected exception (or any of its subclass), test case passes.
- Since, JUnit need to catch the exception and then compare, that's why it expect 2nd parameter as Executable.

assertThrows(Class<T> expectedType, Executable executable)

```
@Test
void testAssertThrows() {

    assertThrows(Exception.class, () -> Integer.parseInt( s: "abc"));
}
```

Exception thrown is its subclass

or, we can directly specific the exact exception type (in above one i defined the parent class exception)

```
@Test
void testAssertThrows() {

    assertThrows( NumberFormatException.class, () -> Integer.parseInt( s: "abc"));
}
```

```
MyServiceTest (com.conce 7 ms
    testAssertThrows()  7 ms

Tests passed: 1 of 1
/Library/Java/Jav

Process finished
```

In below test case, expected exception is "NullPointerException" but actual exception thrown is different, hence the test case failed.

```
@Test
void testAssertThrows() {

    assertThrows( NullPointerException.class, () -> Integer.parseInt( s: "abc"));
}

MyServiceTest (com.conc 10 ms
    testAssertThrows()  10 ms

Tests failed: 1 of 1 test – 10 ms
/Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home

org.opentest4j.AssertionFailedError: Unexpected exception
<4 internal lines>
```

15. **assertThrowsExactly()**

Must throw exact same type exception (no subclasses).

Now, we need to provided the exact exception which is going to be thrown. Like in below i used parent class type, but this test case will failed as exception which is thrown is its Child "NumberFormatException" and it expects the same.

Concept && Coding

```
@Test
void testAssertThrowsExactly() {

    assertThrowsExactly( Exception.class, () -> Integer.parseInt( s: "abc"));
}
```

```
MyServiceTest (com.conc 10 ms
    testAssertThrows()  10 ms
Tests failed: 1 of 1 test – 10 ms
/Library/Java/JavaVirtualMachines/jdk
org.opentest4j.AssertionFailedError:
> <4 internal lines>
```

if we provide the exact same exception, it passes.

```
@Test
void testAssertThrowsExactly() {

    assertThrowsExactly( NumberFormatException.class, () -> Integer.parseInt( s: "abc"));
}
```

```
MyServiceTest (com.conce 7 ms
    testAssertThrowsExactl 7 ms
Tests passed: 1 of 1 test – 7 ms
/Library/Java/JavaVirtualMachines/j

Process finished with exit code 0
```

16. **assertDoesNotThrow()**

Expectation is that method do not throw any exception.

```
@Test
void testAssertDoNotThrow() {

    assertDoesNotThrow(() -> Integer.parseInt( s: "123"));
}
```

```
MyServiceTest (com.conce 7 ms
    testAssertDoNotThrow( 7 ms
Tests passed: 1 of 1 test – 7 ms
/Library/Java/JavaVirtualMachines/

Process finished with exit code 0
```

if any exception is thrown, test case will fail.

```
@Test
void testAssertDoNotThrow() {

    assertDoesNotThrow(() -> Integer.parseInt( s: "abc"));
}
```

```
MyServiceTest (com.conc 10 ms
    testAssertDoNotThrow 10 ms
Tests failed: 1 of 1 test – 10 ms
/Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home/bin/ja
org.opentest4j.AssertionFailedError: Unexpected exception thrown:
> <5 internal lines>
```

17. **assertTimeout()**

Ensures code finishes **within time**, else test fails (but still runs to end).

assertTimeout (Duration timeout, Executable executable)

Concept && Coding

```
@Test
void testAssertTimeout() {

    assertTimeout(Duration.ofMillis(100), () -> Thread.sleep( millis: 50));
}
```

```
✓ MyServiceTest (com.conc 59 ms        ✓ Tests passed: 1 of 1 test – 59 ms
  ✓ testAssertTimeout()   59 ms        /Library/Java/JavaVirtualMachines/

                                       Process finished with exit code 0
```

Test case is taking more time than expected, so it will fail.

```
@Test
void testAssertTimeout() {

    assertTimeout(Duration.ofMillis(100), () -> Thread.sleep( millis: 150));
}
```

```
✗ MyServiceTest (com.con 164 ms        ✗ Tests failed: 1 of 1 test – 164 ms
  ✗ testAssertTimeout()   164 ms       /Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home/bin/

                                       org.opentest4j.AssertionFailedError: execution exceeded timeout
```

18. **assertTimeoutPreemptively()**

- Similar to **assertTimeout()** but abort execution immediately when timeout exceeds.
- Internally it uses different thread for execution, so when the timeout exceeds its interrupts it.

19. **assertAll()**

- Groups multiple assertions, runs all and reports failures together.
  assertAll(Executable...executables)

```
@Test
void testAssertAll() {

    String str = "JUnit5";
    assertAll(
            () -> assertEquals( expected: 6, str.length()),
            () -> assertTrue(str.startsWith("J")),
            () -> assertFalse(str.isEmpty())
    );
}
```

```
✓ MyServiceTest (com.conce 7 ms        ✓ Tests passed: 1 of 1 test – 7 ms
  ✓ testAssertAll()   7 ms             /Library/Java/JavaVirtualMachines/
```

Concept && Coding

It properly shows the result of each assertion:

```
@Test
void testAssertAll() {

    String str = "";
    assertAll(
            () -> assertEquals( expected: 6, str.length()),
            () -> assertTrue(str.startsWith("J")),
            () -> assertFalse(str.isEmpty())
    );
}
```

```
MyServiceTest (com.concu 11 ms
    testAssertAll()  11 ms

Tests failed: 1 of 1 test – 11 ms
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)


org.opentest4j.MultipleFailuresError: Multiple Failures (3 failures)
    org.opentest4j.AssertionFailedError: expected: <6> but was: <0>
    org.opentest4j.AssertionFailedError: expected: <true> but was: <false>
    org.opentest4j.AssertionFailedError: expected: <false> but was: <true>
```

Concept && Coding